

SC2001 PROJECT 1

Ng Zheng Ning
Nguyen Xuan Hoang
Ng Yee Teck



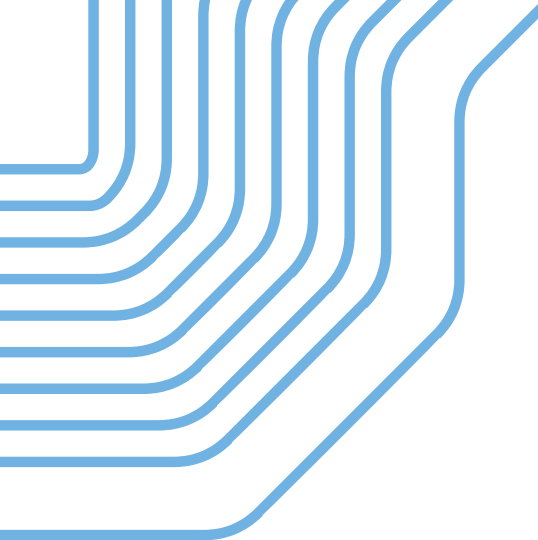


TABLE OF CONTENTS:

- I Prelude
 - I Hybrid Sort Implementation
 - I Data Generation
 - I Results Analysis
 - I
- 

PRELUDE: MERGESORT

- Divide the array until the sub-arrays are size 1 or less
- Recursively merge sub-arrays back together while sorting them
- Time Complexity: $O(n \log n)$

```
def merge_sort(numArray: list[int], start: int = 0, end: int = -1) -> int:
    """
    In-place merge sort that arranges array elements in ascending order.

    Args:
        numArray: Target array to be sorted.
        start: The left bound of values for function to focus on.
        end: The right bound of values for function to focus on.

    Returns:
        The number of key comparisons made in the merge sort process.
    """
    # Initialises the start & end values, if they do not exist.
    if end == -1:
        end = len(numArray) - 1

    # Prematurely ends function if left bound exceeds right.
    if end <= start:
        return 0

    # Splits the array into 2 sections and implement merge sort on each section.
    mid = start + (end - start) // 2
    kcomp_l = merge_sort(numArray, start, mid)
    kcomp_r = merge_sort(numArray, mid + 1, end)

    # Sorts every element in the array concurrently in both sections.
    kcomp_m = merge(numArray, start, mid, end)

    return kcomp_r + kcomp_m + kcomp_l
```

Image: Main Merge Sort function

PRELUDE: MERGESORT (CONT)

Images: Merge Function in Merge Sort

```
def merge(arr: list[int], l: int, m: int, r: int) -> int:
    """
    Helper function for merge sorting.

    Args:
        arr: Array to be sorted.
        l: Left bound.
        m: Middle point.
        r: Right bound.

    Returns:
        Number of key comparisons made during sorting process.
    """
    # Initialises a sub-array with the left and right bounds, as well as pointers and
    # kcomp to track key comparisons.
    temp = arr[l:r+1]
    i, j, left_end, right_end, k = 0, m + 1 - l, m - l, r - l, l
    kcomp = 0
```

```
    # Iterates through the temp array, adding elements in ascending order to final array.
    # Overwrites array values in the original array with the next smallest element per loop.
    while i <= left_end and j <= right_end:
        kcomp += 1
        if temp[i] <= temp[j]:
            arr[k] = temp[i]
            i += 1
        else:
            arr[k] = temp[j]
            j += 1
        k += 1

    # Adds all remaining elements (if any) to the final array.
    while i <= left_end:
        arr[k] = temp[i]
        i += 1
        k += 1
    while j <= right_end:
        arr[k] = temp[j]
        j += 1
        k += 1

    return kcomp
```

PRELUDE: INSERTION SORT

- Start with first array element in “sorted” portion
- Iteratively walks through “unsorted” portion of array to insert them into “sorted” part accordingly
- Worst case: $O(n^2)$ (reverse sorted array)
- Best case: $O(n)$ (sorted array)

```
def insertion_sort(arr: list[int], l: int = 0, r: int = -1) -> int:
    """
    Uses insertion sort to arrange array elements in ascending order.

    Args:
        arr: Number array to be sorted.
        l: Left bound of unsorted elements.
        r: Right bound of unsorted elements.

    Returns:
        Number of key comparisons made.
    """
    # Initialises end value if they do not exist and kcomp to track key comparisons.
    kcomp = 0

    if r == -1:
        r = len(arr) - 1

    # Insertion sort main function but to shift elements the function uses
    # overwriting compared to swapping.
    for i in range(l + 1, r + 1):
        j = i - 1
        tmp = arr[i]
        while j >= l:
            kcomp += 1
            if arr[j] > tmp:
                arr[j + 1] = arr[j]
                j -= 1
            else:
                break
        arr[j + 1] = tmp

    return kcomp
```

Image: Insertion Sort

PART A: HYBRID SORT

- Combination of merge & insertion sort
- Every execution of merge sort function checks if array length exceeds threshold S
- Arrays / sub-arrays of size S or smaller are sorted using insertion sort

PART A: HYBRID SORT

```
1 def hybrid_sort(numArray: list[int], S: int, start: int = 0, end: int = -1) -> int:
2     """
3     Implements a hybrid sort that uses merge sort on a number array by default.
4     If array size goes below S, uses insertion sort instead.
5     Sorts in ascending order.
6
7     Args:
8         arr: Number array to be sorted.
9         S: The threshold for deciding the switch between insertion & merge sort.
10        start: The left bound of values for function to focus on.
11        end: The right bound of values for function to focus on.
12
13    Returns:
14        Number of key comparisons made.
15    """
16    # Initialises the start & end values, if they do not exist.
17    if end == -1:
18        end = len(numArray) - 1
19
20    # Prematurely ends function if left bound exceeds right.
21    if end <= start:
22        return 0
23
24    # Runs an insertion sort algorithm for arrays smaller or equal to S.
25    # Note that an array with start index l and end index r has r - l + 1 elements.
26    # Terminates prematurely when done.
27    if (end - start + 1) <= S:
28        return insertion_sort(numArray, start, end)
29
30    # Splits the array into 2 sections and implement merge sort on each section.
31    mid = start + (end - start) // 2
32    kcomp_l = hybrid_sort(numArray, S, start, mid)
33    kcomp_r = hybrid_sort(numArray, S, mid + 1, end)
34
35    # Sorts every element in the array concurrently in both sections.
36    kcomp_m = merge(numArray, start, mid, end)
37
38    return kcomp_r + kcomp_m + kcomp_l
```

PART B: DATA GENERATION

- Generation of arrays with sizes from $1e3$ to $1e7$
- Maximum permissible value in datasets is $1e8 + 5$
- Benchmarking (Part (c) iii and (d)) done in C++ for better efficiency & accuracy in measuring algorithm runtime.

```
import math
import os
from pathlib import Path
import random

MAX_VAL = int(1e8) + 5
SEED = 42

def generate_csv_dataset(dir_path: str, filename: str, count: int, seed: int = SEED, max_val: int = MAX_VAL) -> None:
    """
    Generates a CSV file containing random integers.

    Args:
        dir_path: Name of the output directory.
        filename: Name of the output CSV file (e.g., "data_10m.csv").
        count: Number of random numbers to generate.
        seed: Random seed value.
        max_val: Maximum random integer value.

    Returns:
        None (creates the .csv files)
    """
    target_dir = Path(dir_path)
    target_dir.mkdir(parents=True, exist_ok=True)
    full_path = target_dir / filename

    # Check if file already exists
    if full_path.exists():
        print(f'{dir_path}/{filename}' already exists. Skipping.")
        return

    # Use python's Built-in Random instance for seeded generation
    rng = random.Random(seed)

    # Fast batch writing using a list buffer to minimize file I/O operations
    batch_size = 65536

    # 1MB buffering on file output matching standard C++ stream buffer behavior
    with open(full_path, "w", encoding="utf-8", buffering=1024 * 1024) as file:
        file.write("value\n")

        written = 0
        while written < count:
            current_batch_size = min(count - written, batch_size)

            # Generate and format a batch of numbers in memory
            batch = [str(rng.randint(1, max_val)) for _ in range(current_batch_size)]

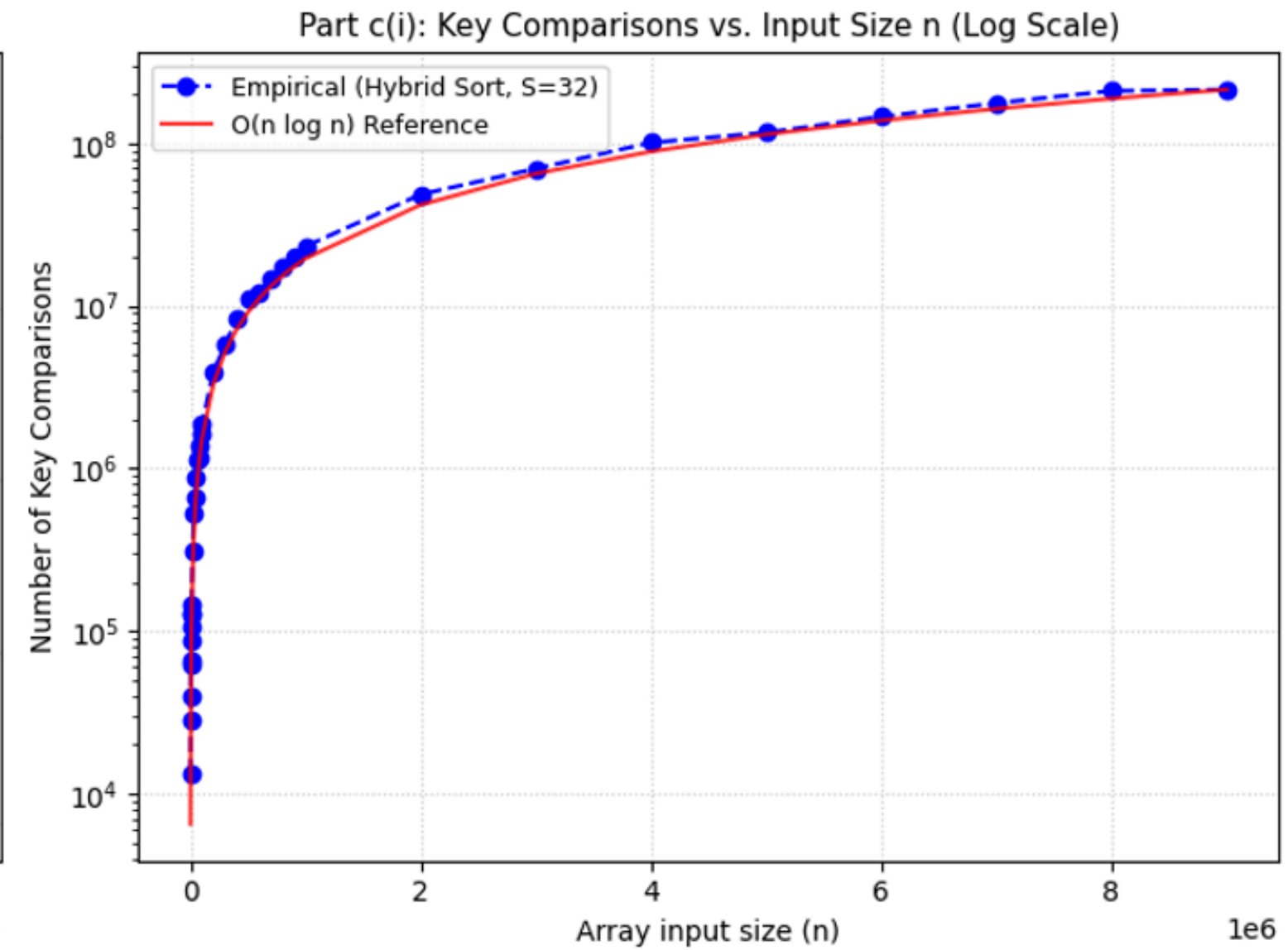
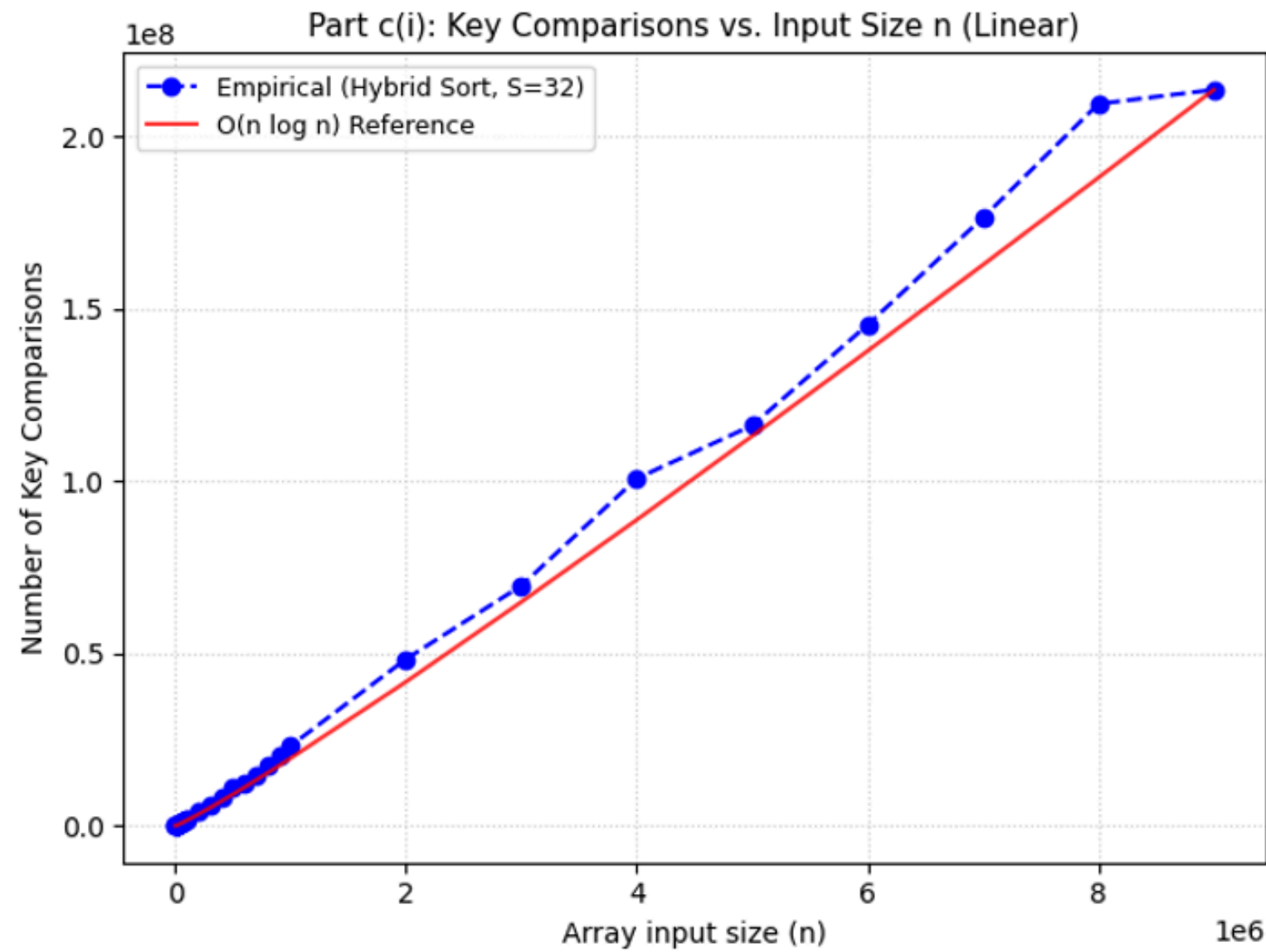
            # Write out buffered text block efficiently
            file.write("\n".join(batch) + "\n")
            written += current_batch_size

    print(f"Successfully generated {count} values in '{filename}')
```

PART C:

i. Fixed S, vary n

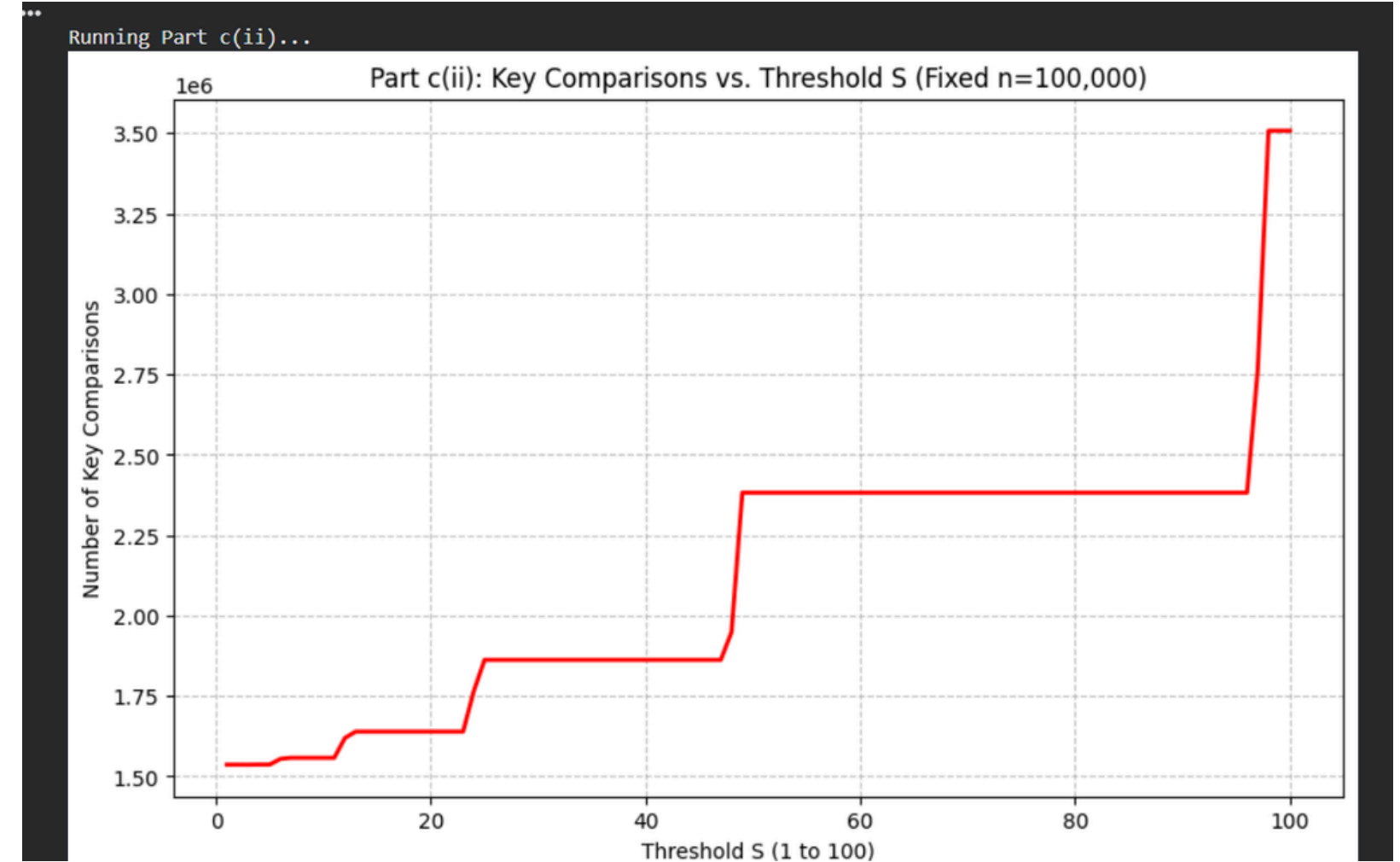
- Chosen S to be fixed at 32
- log y axis (right)



PART C:

ii. Fixed n, vary S

- Key comparisons against different values of S with fixed n
- Values of S range from 1 to 100

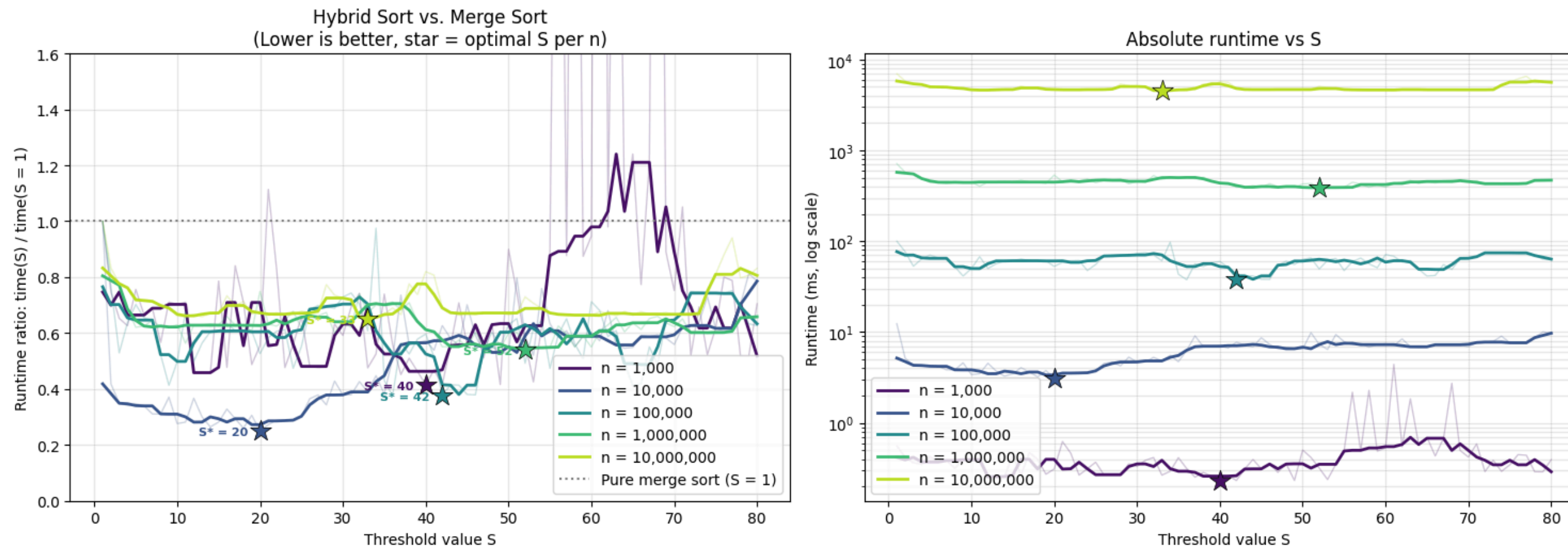


PART C:

iii. Find Optimal S

We measure the runtime of Hybrid Sort with fixed n while varying S from 1 to 80, for input sizes from $1e3$ up to $1e7$. To make the data less noisy, we applied a light rolling-median trend line so that the true optimum is visible. Optimal values of S is the value of S with the fastest measured runtime, marked with a star.

(c)(iii) Determining the optimal threshold S for the hybrid algorithm



PART C:

iii. Find Optimal S

- The final value of S is not a constant, but lands in the ranges of 20 – 50.
- For every input size, hybrid sort is faster than merge sort by 1.5x – 4x.
- While benchmarking S over key comparisons instead of runtime, S is approximately 3 – 4.
- This aligns with the project introduction that hybrid sort wins because it has less recursive overhead.

Input size n	Optimal S*	Best runtime	Runtime at S=1 (merge sort)	Speedup
1,000	40	0.232 ms	0.560 ms	~2.4×
10,000	20	3.09 ms	12.36 ms	~4.0×
100,000	42	37.69 ms	100.26 ms	~2.7×
1,000,000	52	386.11 ms	713.15 ms	~1.8×
10,000,000	33	4,532.39 ms	6,958.81 ms	~1.5×

PART D: MERGE SORT VS HYBRID SORT AT $N = 1E7$

We set $S = 32$, 1 away from optimal $S (=33)$ found in last slide for $n = 1e7$

Hybrid Sort has 10% more key comparisons than Merge Sort, but runs 33% faster.

